

AI Ops Board 기획 및 구현 요약



업로드해주신 PDF는 AI 코딩 및 로컬 LLM 운용을 위한 실전 정보 자동 큐레이션 시스템 (AI Ops Board) 구축에 관한 상세 기획과 구현 과정을 담고 있습니다.

단순한 뉴스 전달을 넘어, ****최소 비용(토큰)으로 최대 효율****을 뽑아내기 위한 운영 전략과 이를 자동화하는 시스템 구조가 핵심입니다. 내용을 5가지 핵심 영역으로 정리해 드립니다.

1. 프로젝트 정의: AI Ops Board

이 프로젝트는 AI 코딩과 LLM 운용에 관한 파편화된 정보를 수집하여 ****실행 가능한 가이드****와 ****최적의 셋팅****으로 변환해 주는 자동 큐레이션 게시판입니다.

- 핵심 철학:** 정보 수집보다 중요한 것은 실전성 필터링이며, AI를 많이 쓰는 것보다 '잘' 쓰게 만드는 것이 목적입니다.
- 차별점:** 일반 뉴스가 "이런 모델이 나왔다"를 말할 때, 이 보드는 "이 모델을 지금 내 환경에 어떻게 써야 성능이 올라가는지"를 알려줍니다.

2. 핵심 운영 전략 (토큰 절약 및 효율화)

가장 강조된 부분으로, AI 도구를 계층별로 나누어 사용하는 **'역할 분리'** 전략입니다.

구분	도구 (예시)	주요 역할	전략
작업자 (Worker)	OpenCode, 로컬 모델	파일 탐색, 로그 요약, 단순 구현	전체 탐색
시니어 (Senior)	Claude Code	복잡한 아키텍처 판단, 버그 분석, 최종 리뷰	diff 급 판
□			

- AGENTS.md / CLAUDE.md 관리:** 규칙 파일은 100~200줄 이하로 짧게 유지하며, ****영어(모델용)****와 ****한국어(사람용)****를 병기하는 구조가 가장 효율적입니다.

- **작업 흐름:** 탐색(Explore) → 구현(Build) → 리뷰(Review) 순으로 단계를 쪼개어 hallucination을 방지하고 토큰을 아킵니다.
-

3. 시스템 아키텍처 및 기술 스택

실제 서비스 구현을 위한 기술적 구성 요소입니다.

- **Backend:** FastAPI 기반의 비동기 서버.
 - **Database:** PostgreSQL(데이터 저장), Redis(작업 큐).
 - **Data Pipeline:**
 - **수집:** RSSHub, Reddit API, GitHub Trending, Hacker News .
 - **추출:** `trafilatura` (본문 및 메타데이터 추출), `Playwright` (JS 사이트 대응).
 - **분석/요약:** 로컬 VLLM 또는 OpenAI 호환 API를 통한 한국어 요약 및 실전성 점수 산정.
 - **LLM Router:** 요청의 난이도에 따라 로컬 무료 모델과 유료 모델을 자동으로 분기하여 비용을 최적화합니다.
-

4. 게시판 및 콘텐츠 구조

사용자가 정보를 즉시 실행에 옮길 수 있도록 설계된 카테고리 구조입니다.

- **오늘의 추천 셋팅 (최상단):** 도메인별(게임, 프론트, 백엔드 등) 최적의 AI 운용 조합을 매일 1개 선정하여 노출합니다.
 - **주요 카테고리:**
 - **실전 피드:** 레딧/웹 기반의 실제 사용 후기 및 모델 비교.
 - **잘 쓰는 방법 (How-To):** 나쁜 패턴과 좋은 패턴을 비교하여 바로 적용 가능한 가이드 제공 .
 - **플러그인/MCP 추천:** AI의 기능을 확장하는 모델 컨텍스트 프로토콜(MCP) 서버 추천 및 보안 주의사항 .
 - **주의/함정:** 시행착오를 줄이기 위해 "하면 망하는 것들" 정리 .
-

5. 단계별 구현 로드맵 (MVP 개발)

총 5단계에 걸쳐 프로토타입에서 정식 서비스로 확장하는 계획입니다 .

1. **1단계 (기반 구축)**: RSS, HN, GitHub 수집기 및 DB 스키마 설계.
2. **2단계 (데이터 정제)**: 본문 추출 도구 통합 및 수집 자동화.
3. **3단계 (지능화)**: LLM을 활용한 요약, 태깅, 실전성 점수화 루틴 개발.
4. **4단계 (UI/UX)**: React/TS 기반의 대시보드 및 게시판 프론트엔드 구현.
5. **5단계 (최적화)**: 매일 아침 "오늘 볼만한 브리핑" 생성 및 유저 참여 기능(팁 공유 등) 추가.

💡 **핵심 인사이트**: 이 프로젝트는 단순히 뉴스를 읽는 곳이 아니라, **"내 PC와 환경에 맞는 최적의 AI 코딩 운용법을 매일 자동으로 업데이트해 주는 대시보드"**를 지향합니다.

더 구체적으로 알고 싶은 부분(예: 특정 카테고리의 DB 설계나 AGENTS.md 템플릿 등)이 있다면 말씀해 주세요! 한 단계 더 깊게 들어가 볼까요?



프로젝트의 성격과 문서에서 강조된 핵심 키워드를 바탕으로 프로젝트 이름을 제안해 드릴게요. 문서 내에서 이미 추천된 이름들과 주요 컨셉을 연결해 보았습니다.

1. 문서 추천 이름 (이미 검토된 후보)

문서에서 프로젝트 정의 단계에서 직접적으로 추천된 이름들입니다.

- **AI Ops Board**: AI 운영 최적화라는 직관적인 의미를 담고 있습니다.
- **LLM Ops Daily**: 매일 갱신되는 LLM 운용 정보라는 특성을 강조합니다.
- **AgentOps Feed**: 에이전트와 MCP 중심의 수집기라는 느낌을 줍니다.

2. 서비스 핵심 기능 강조형

이 서비스의 가장 강력한 기능인 **"오늘의 최강 셋팅"**과 **"실전성"**에 집중한 이름입니다.

- **오늘의 추천 셋팅 (Today's Best Setup)**: 문서 마지막 단계에서 최종적으로 제안된 이름으로, 사용자에게 가장 필요한 결론을 제공한다는 의미입니다.
- **실전 AI 보드 (Practical AI Board)**: 단순 뉴스가 아닌 '실전 정보'와 '실행 가능한 가이드'임을 강조합니다.
- **Token Saver (토큰 세이버)**: 프로젝트의 큰 목표 중 하나인 '최소 토큰 사용'과 '비용 절감'을 브랜드화한 이름입니다.

3. 워크플로우 및 아키텍처 중심형

AI를 단순 도구가 아닌 **AI 시스템**과 **AI 운영**의 관점으로 보는 이름들입니다.

- **AI 운영 최적화 게시판**: 프로젝트 제안서에서 명시된 공식적인 목적을 그대로 담은 이름입니다.
- **Dev-AI Ops**: 개발(Dev)과 AI 운영(Ops)을 결합하여 전문적인 느낌을 줍니다.
- **Agent Guidebook (에이전트 가이드북)**: AGENTS.md와 CLAUDE.md 등 에이전트 운용 규칙을 제공하는 정체성을 나타냅니다.

개인적인 추천을 드리자면, 문서 후반부에 언급된 **"오늘의 추천 셋팅"**이 가장 매력적이에요. 정보가 너무 많아 결정이 힘든 개발자들에게 "오늘의 정답"을 딱 집어준다는 느낌이 강하거든요.

혹시 이 중에서 마음에 드는 방향이 있으신가요, 아니면 조금 더 짧고 힙한 느낌을 원하시나요?



문서에서 제안 및 최종 결정된 프로젝트 이름들을 정리해 드립니다.

1. 공식 프로젝트명 (추천 후보)

프로젝트 정의 단계에서 가장 먼저 추천된 이름들입니다.

- **AI Ops Board**: AI 운영 최적화 게시판이라는 의미로 가장 비중 있게 언급되었습니다.
- **LLM Ops Daily**: 매일 업데이트되는 LLM 운용 정보라는 특성을 강조합니다.
- **AgentOps Feed**: 에이전트 운용과 관련된 피드 공급이라는 의미를 담고 있습니다.

2. 서비스 핵심 브랜드명

서비스의 킬러 콘텐츠이자 대시보드 타이틀로 제안된 이름입니다.

- **오늘의 추천 셋팅**: 데이터 구조를 정리하며 프로덕션급으로 갈아엎을 때 최종적으로 확정된 이름입니다.
- **오늘의 셋팅 최강**: 도메인별 최적의 운용 조합을 보여주는 메인 배너의 명칭입니다.

3. 프로젝트의 정의 및 성격

제안서에 명시된 프로젝트의 공식적인 성격입니다.

- **AI 운영 최적화 게시판**: 프로젝트의 목적을 명확히 드러내는 설명적 명칭입니다.

요약하자면: 프로젝트의 전체 시스템 명칭은 **AI Ops Board**로, 사용자가 마주하는 서비스의 메인 이름은 **오늘의 추천 셋팅**으로 기획되었습니다.



정확합니다! 아주 날카로운 지적이세요. 사실 이 시스템의 설계 철학을 관통하는 통찰이기도 합니다.

말씀하신 대로 **'**'토큰 최적화'** 는 **'**'잘 쓰는 방법(운용법)'** 이라는 큰 주머니 안에 담긴 가장 핵심적인 알맹이입니다. 문서에서도 이 둘을 별개로 보지 않고, 다음과 같은 논리로 연결하고 있어요.

1. 운용 구조가 곧 토큰 절약이다

- **기술이 아니라 구조:** 토큰 절약은 단순한 '프롬프트 기술'이 아니라, 모델의 역할을 분리하는 '운용 구조'에서 나옵니다.
- **비용이 곧 실력:** AI를 "잘 쓴다"는 것은 결국 최소한의 정보로 정확한 판단을 끌어내어 비용(토큰)을 최적화하는 것과 같습니다.

2. '잘 쓰는 방법' 카테고리의 핵심 지표

이 보드에서 '잘 쓰는 방법'이라는 카테고리를 따로 강조한 이유는, 단순히 "이 기능을 써보세요"라고 소개하는 뉴스 사이트와 차별화하기 위해서입니다.

- **목표의 일치:** '잘 쓰는 방법'의 궁극적인 목표는 **최소 토큰 + 최대 효율**입니다.
- **실전 가이드화:** "Claude Code에게는 전체 스캔을 시키지 마라"는 운용 규칙은, 결국 토큰 폭발을 막기 위한 실전 스킬입니다.

3. 역할 분리를 통한 최적화 (Practical Example)

문서에서 제안한 가장 대표적인 '잘 쓰는 법'이자 '토큰 최적화' 사례는 다음과 같습니다.

구분	잘 쓰는 방법 (운용법)	토큰 최적화 결과
OpenCode (로컬)	파일 탐색, 로그 요약 등 단순 노동 담당	비싼 클라우드 모델의 컨텍스트 소모 방지
Claude Code (유료)	복잡한 판단 및 최종 diff 리뷰 브라슈해	꼭 필요한 고난도 작업에만 토큰 지출